المكدسات (Stacks)

🎯 المفهوم الأساسي

تعريف المكدس (Stack)

المكدس هو هيكل بيانات يعمل بمبدأ **LIFO**: Last-In, First-Out

"آخر ما يدخل، أول ما يخرج"

تشبيه بسيط 🍟

تخيل **كومة أطباق** أو **علبة Pringles**:

- تضيف طبق جديد ← يصير فوق (Push)
- تاخذ طبق ← لازم تاخذ اللي فوق (Pop)
- **مستحيل** تسحب الطبق من النص أو من الأسفل!

📋 العمليات الأساسية (Stack Operations)

العملية	الوصف	التعقيد الزمني	ملاحظات
`push(e)`	إضافة عنصر في الأعلى	O(1)	–
`pop()`	حذف العنصر العلوي	O(1)	⚠️ لا يُرجع القيمة
`top()`	عرض العنصر العلوي	O(1)	بدون حذف
`size()`	عدد العناصر	O(1)	–
`empty()`	هل المكدس فارغ؟	O(1)	يُرجع true/false

⚠️ تحذير مهم جداً!

```cpp
// ❌ خطأ شائع
s.pop();  // إذا كان المكدس فارغ → Undefined Behavior

// ✅ الطريقة الصحيحة
if (!s.empty()) {
    int value = s.top();  // احفظ القيمة أولاً
    s.pop();              // ثم احذف
}
```

🌟 التطبيقات العملية

1. زر "الرجوع" في المتصفح 🌐

```text
زرت google.com  → Push
زرت youtube.com → Push
زرت facebook.com → Push
ضغطت "Back" → Pop (facebook)
ضغطت "Back" → Pop (youtube)
```

2. التراجع (Undo) في محررات النصوص ✏️

```text
كتبت "Hello" → Push
كتبت " World" → Push
Ctrl+Z → Pop (حذف " World")
```

3. عكس المتجه (Reversing)

```cpp
// عكس كلمة "ALI"
push('A');  push('L');  push('I');
pop() → 'I'
pop() → 'L'
pop() → 'A'
النتيجة: "ILA"
```

4. مطابقة الأقواس والأكواد 🔍

```cpp
تعبير: ([2 * 3] + 5)
الأقواس متطابقة ✅

تعبير: (2 * 3] + 5)
خطأ: قوس مربع غير مُغلق ❌
```

🏗️ طرق التنفيذ (Implementation Methods)

الطريقة الأولى: باستخدام المصفوفة (Array-Based)

البنية الأساسية

```cpp
class ArrayStack {
private:
    int* S;       // المصفوفة
    int t;        // مؤشر العنصر العلوي (Top)
    int N;        // السعة القصوى
public:
    ArrayStack(int capacity);
};
```

آلية العمل

- **البداية**: `t = -1` (المكدس فارغ)
- **الحجم**: `size() = t + 1`
- **Push**: زيادة `t` ثم إضافة العنصر في `S[t]`
- **Pop**: تقليل `t` بمقدار 1

مثال تفصيلي

```cpp
// في البداية
t = -1  → المكدس فارغ

// بعد push(5)
t = 0, S[0] = 5

// بعد push(10)
t = 1, S[1] = 10

// بعد push(15)
t = 2, S[2] = 15

// بعد pop()
t = 1  (العنصر 15 أصبح خارج النطاق)
```

المميزات ✅

- **سريع جداً**: جميع العمليات O(1)
- **بسيط في التنفيذ**
- **استهلاك ذاكرة قليل** (لا توجد pointers)

العيوب ❌

- **حجم ثابت**: لازم تحدد السعة من البداية
- **Stack Overflow**: إذا امتلأت المصفوفة ترمي استثناء `StackFull`
- **هدر الذاكرة**: إذا حجزت 1000 واستخدمت 10 فقط

الطريقة الثانية: باستخدام القائمة المتصلة **(Linked List)**

البنية الأساسية

```cpp
class LinkedStack {
private:
    SLinkedList<int> list;  // قائمة متصلة أحادية
public:
    void push(int e) { list.addFront(e); }
    void pop() { list.removeFront(); }
    int top() { return list.front(); }
};
```

آلية العمل

- **رأس القائمة (Head) = أعلى المكدس (Top)**
- `Push()` → `addFront` - إضافة في الرأس
- `Pop()` → `removeFront` - حذف من الرأس

لماذا الرأس وليس الذيل؟ 🤔

```text
الرأس (Head):
- addFront: O(1) ✅
- removeFront: O(1) ✅

الذيل (Tail):
- addBack: O(1) ✅
- removeBack: O(n) ❌ (نحتاج نوصل للعنصر قبل الأخير)
```

المميزات ✅

- **حجم ديناميكي**: ينمو حسب الحاجة
- **لا يوجد Stack Overflow**
- **استغلال أمثل للذاكرة**

العيوب ❌

- **ذاكرة إضافية**: كل عقدة تحتاج pointer
- **أبطأ قليلاً**: بسبب تخصيص الذاكرة الديناميكي

📊 جدول المقارنة السريع

الميزة	Array-Based	Linked List
السرعة	⭐⭐⭐⭐⭐ (الأسرع)	⭐⭐⭐⭐ (سريع)
الحجم	ثابت ومحدود	ديناميكي ومرن
الذاكرة	أقل استهلاك	أكثر استهلاك
الأمان	Stack Overflow	آمن
الاستخدام	للأحجام المعروفة	للأحجام المتغيرة

🎹 خوارزمية مطابقة الأقواس (Matching Parentheses)

المشكلة

التحقق من صحة الأقواس في تعبير رياضي أو كود HTML.

الخوارزمية خطوة بخطوة

1. **البداية والتهيئة:**
   - أنشئ مكدساً فارغاً (Stack) وسمّه **S**.
2. **عملية المسح:**
   - اقرأ "التعبير (expression) "رمزاً تلو الآخر من اليسار إلى اليمين.
3. **التكرار (لكل رمز):**
   - **أ) حالة رمز الفتح:**
     - إذا كان الرمز هو (أو [أو { أو <tag>:
     - قم بعملية دفع (**Push**) لهذا الرمز إلى المكدس **S**.
   - **ب) حالة رمز الإغلاق:**
     - إذا كان الرمز هو) أو] أو } أو </tag>:
       - **تحقق من المكدس:**
       - إذا كان المكدس **S** فارغاً ❌ :**خطأ** (لا يوجد قوس فتح سابق ليطابقه).
       - إذا كان المكدس **S** غير فارغ:
         - قم بعملية سحب (**Pop**) للعنصر العلوي من المكدس.
         - **قارن النوعين:**
         - إذا تطابق رمز الفتح المسحوب مع رمز الإغلاق الحالي ✅ :**استمر**.
         - إذا لم يتطابقا ❌ :**خطأ** (عدم توافق الأقواس).
4. **النهاية (بعد اكتمال المسح):**
   - **تحقق من حالة المكدس:S**
     - إذا كان المكدس فارغاً ✅ :**التعبير صحيح**.
     - إذا لم يكن فارغاً ❌ :**خطأ** (توجد أقواس مفتوحة لم تُغلق).

مثال اختبار صحة التعبير ([2 * 3] + 5) :

1. **الخطوة الأولى:**

- **الرمز المقروء :** (
- **الإجراء :**رمز فتح ← إضافته للستاك.(Push)
- **محتويات الستاك:** [(

2. **الخطوة الثانية:**

- **الرمز المقروء:** [
- *(ملاحظة: يتم تجاهل الأرقام والعمليات الحسابية)*
- **الإجراء :**رمز فتح ← إضافته للستاك.(Push)
- **محتويات الستاك:** [[, (

3. **الخطوة الثالثة:**

- **الرمز المقروء:** [
- **الإجراء :**رمز إغلاق ← سحب (Pop) ومقارنة.
- **التحقق :**العنصر المسحوب [يطابق [.
- **محتويات الستاك:** [(

4. **الخطوة الرابعة:**

- **الرمز المقروء:** (
- **الإجراء :**رمز إغلاق ← سحب (Pop) ومقارنة.
- **التحقق :**العنصر المسحوب (يطابق (.
- **محتويات الستاك: فارغ**

النتيجة النهائية:

- انتهى التعبير والستاك (Stack) فارغ.
- ✅ **التعبير صحيح.**

التعقيد الزمني للخوارزمية

- O(n) حيث n هي طول التعبير
- نمسح كل رمز مرة واحدة فقط
- كل عملية Push/Pop تأخذ O(1)

💻 القواعد البرمجية المهمة

1. قاعدة الأمان الأساسية ⚠️

```cpp
// ❌ خطير: STL لا ترمي استثناء!
stack<int> s;
s.pop();  // Undefined Behavior إذا فارغ

// ✅ الطريقة الآمنة دائماً
if (!s.empty()) {
    int val = s.top();
    s.pop();
}
```

2. قاعدة حفظ القيمة قبل الحذف

```cpp
// ❌ خطأ: pop لا تُرجع القيمة
int x = s.pop();  // خطأ تصريفي (Compile Error)

// ✅ صحيح
int x = s.top();  // احفظ القيمة
s.pop();          // ثم احذف
```

3. قاعدة التحقق من الامتلاء (Array-Based)

```cpp
void push(int e) {
    if (size() == N) {
        throw StackFull();  // رمي استثناء
    }
    S[++t] = e;
}
```

4. قاعدة التحقق من الفراغ

```cpp
int top() const {
    if (empty()) {
        throw StackEmpty();
    }
    return S[t];
}
```

5. قاعدة const للدوال القارئة

```cpp
// هذه الدوال لا تُغير المكدس
int size() const { return t + 1; }
bool empty() const { return t < 0; }
int top() const { return S[t]; }
```

📝 ملخص سريع للمراجعة

المفاهيم الأساسية

- **LIFO**: آخر داخل، أول خارج
- **Top**: العنصر العلوي (نقطة الإدخال والإخراج)
- **جميع العمليات O(1)**: السرعة الفائقة

التنفيذ

Array	Linked List
ثابت الحجم	ديناميكي
`t = -1` في البداية	Head = Top
`size = t + 1`	`addFront/removeFront`

الاستثناءات

- **StackEmpty**: عند Pop/Top على مكدس فارغ
- **StackFull**: عند Push على مصفوفة ممتلئة

التطبيقات الرئيسية

1. Back/Undo
2. عكس البيانات
3. مطابقة الأقواس O(n)
4. تقييم التعابير الرياضية

الطابور (Queue)

🎯 المفهوم الأساسي

تعريف الطابور (Queue)

الطابور هو هيكل بيانات يعمل بمبدأ **FIFO**: First-In, First-Out

"أول ما يدخل، أول ما يخرج"

تشبيه بسيط 🏪

تخيل **طابور المخبز** أو **طابور البنك**:

- الشخص الأول الواصل → أول واحد يُخدم ويطلع (Front)
- الشخص الجديد → يوقف بالنهاية (Rear)
- **مستحيل** اللي وصل أخيراً يطلع قبل اللي قبله!

🔄 الفرق بين Stack و Queue

الميزة	Stack	Queue
المبدأ	LIFO (Last-In, First-Out)	FIFO (First-In, First-Out)
التشبيه	كومة أطباق	طابور مخبز
الإدخال	من الأعلى (Top)	من الخلف (Rear)
الإخراج	من الأعلى (Top)	من الأمام (Front)
نقاط الوصول	نقطة واحدة	نقطتان (Front & Rear)

مثال توضيحي عن الفرق بين العملية الاساسية (FILO و FIFO):

Stack (FILO): [1, 2, 3, 4, 5]
push(6) → [1, 2, 3, 4, 5, 6]
pop() → يُزيل 6 (آخر عنصر)

Queue (FIFO): [1, 2, 3, 4, 5]
enqueue(6) → [1, 2, 3, 4, 5, 6]
dequeue() → يُزيل 1 (أول عنصر)

📋 العمليات الأساسية (Queue Operations)

العملية	الوصف	التعقيد الزمني	ملاحظات
`enqueue(e)`	إضافة عنصر في المؤخرة (Rear)	O(1)	–
`dequeue()`	حذف العنصر الأمامي (Front)	O(1)	⚠️ لا يُرجع القيمة
`front()`	عرض العنصر الأمامي	O(1)	بدون حذف
`size()`	عدد العناصر	O(1)	const دالة
`empty()`	هل الطابور فارغ؟	O(1)	const دالة

⚠️ تحذيرات مهمة جداً!

1. حفظ القيمة قبل الحذف

```cpp
// ❌ خطأ: dequeue لا تُرجع القيمة
int value = q.dequeue();  // خطأ تصريفي

// ✅ صحيح
int value = q.front();  // احفظ القيمة أولاً
q.dequeue();            // ثم احذف
```

2. التحقق من الفراغ (في STL)

```cpp
// ❌ خطر: STL لا ترمي استثناء!
queue<int> q;
q.pop();  // Undefined Behavior إذا فارغ

// ✅ الطريقة الآمنة
if (!q.empty()) {
    int val = q.front();
    q.pop();
}
```

🌟 الفرق بين Queue ADT و STL Queue

الميزة	Queue ADT (النظري)	STL Queue (C++)
الإضافة	`enqueue(e)`	`push(e)`
الحذف	`dequeue()`	`pop()`
الأمام	`front()`	`front()`
الخلف	–	`back()` ✨ (إضافي)
الاستثناء	يرمي `QueueEmpty`	Undefined Behavior

ميزة إضافية في STL

```cpp
queue<int> q;
q.push(10);
q.push(20);
q.push(30);

cout << q.front();  // 10 (الأول)
cout << q.back();   // 30 (الأخير) ✅
```

🏗️ طرق التنفيذ (Implementation Methods)

الطريقة الأولى: المصفوفة البسيطة (Simple Array) - ❌ غير فعّالة

المشكلة

```text
Initial: [A, B, C, D, _]
         ↑ Front

After dequeue():
[_, B, C, D, _]  → يجب نقل كل العناصر!
   ↑ Front

[B, C, D, _, _]  → O(n) لكل dequeue ❌
↑ Front
```

لماذا فاشلة؟

- كل عملية `dequeue` تحتاج **نقل جميع العناصر** خطوة للأمام
- التعقيد يصبح **O(n)** بدلاً من O(1)
- **غير مقبول** للاستخدام العملي

الطريقة الثانية: المصفوفة الدائرية (Circular Array) - ✅ فعّالة

الفكرة الذكية 💡

بدلاً من نقل العناصر، **نحرك المؤشرات**!

البنية الأساسية

```cpp
class CircularQueue {
private:
    int* Q;       // المصفوفة
    int f;        // مؤشر الأمام (Front)
    int r;        // مؤشر الخلف (Rear)
    int n;        // عدد العناصر
    int N;        // السعة القصوى
};
```

المؤشرات الثلاثة

المؤشر	الوصف	القيمة الابتدائية
f	فهرس العنصر الأمامي	0
r	فهرس الخلية **بعد** المؤخرة	0
n	عدد العناصر الحالي	0

السحر: عامل Modulo (%)

```cpp
// للتحرك خطوة للأمام بشكل دائري
next_index = (current_index + 1) % N

// أمثلة عملية (N = 5):
(0 + 1) % 5 = 1  ✓
(1 + 1) % 5 = 2  ✓
(4 + 1) % 5 = 0  ← يلف للبداية! ✨
```

المميزات ✅

- **جميع العمليات O(1)** - سريع جداً
- **لا نقل للعناصر** - كفاءة عالية
- **بسيط نسبياً** - سهل البرمجة

العيوب ❌

- **حجم ثابت**: لازم تحدد N من البداية
- **Queue Overflow**: إذا امتلأت
- **هدر محتمل**: إذا حجزت 1000 واستخدمت 10

الطريقة الثالثة: القائمة المتصلة الدائرية (Circular Linked List) - ✅ الأمثل

لماذا دائرية وليست عادية؟

```text
قائمة عادية:
[Head] → [A] → [B] → [C] → NULL
  ↑                      ↑
Front                  Rear

المشكلة:
- نحتاج مؤشرين (Head و Tail)
- أو نضحي بالكفاءة

قائمة دائرية:
     Cursor (Rear)
          ↓
    [C] → [A] → [B] → [C] → ...
          ↑
       Front (cursor->next)

الحل الذكي:
- مؤشر واحد فقط! (Cursor على Rear)
- Front = cursor->next تلقائياً ✨
```

المميزات ✅

- **حجم ديناميكي** - ينمو حسب الحاجة
- **لا حد أقصى** - إلا ذاكرة النظام
- **مؤشر واحد** - تصميم أنيق
- **جميع العمليات O(1)** - كفاءة عالية

العيوب ❌

- **ذاكرة إضافية** - لتخزين الـ pointers
- **أبطأ قليلاً** - تخصيص ذاكرة ديناميكي
- **أعقد قليلاً** - يحتاج فهم القوائم الدائرية

📊 جدول المقارنة الشامل

الميزة	Simple Array	Circular Array	Circular Linked List
enqueue	O(1)	✅ O(1)	✅ O(1)
dequeue	❌ O(n)	✅ O(1)	✅ O(1)
الحجم	ثابت	ثابت	ديناميكي ✅
الذاكرة	فعّالة	فعّالة	إضافية (pointers)
التعقيد	بسيط	متوسط	متقدم
الاستخدام	❌ تجنبه	✅ ممتاز	✅ الأفضل

🎓 التطبيقات العملية

1. نظام الطباعة (Print Queue) 🖨️

```text
Document1 → Document2 → Document3 → Document4
    ↑                                   ↑
آخر وثيقة                          يُطبع الآن
```

2. جدولة العمليات (CPU Scheduling) 💻

```text
Process A → Process B → Process C
```

كل عملية تأخذ دورها بالترتيب (Round Robin).

3. BFS في الرسوم البيانية 🕸️

```cpp
void BFS(Graph& g, Node start) {
    queue<Node> q;
    q.push(start);

    while (!q.empty()) {
        Node current = q.front();
        q.pop();

        // زيارة الجيران
        for (Node neighbor : current.neighbors) {
            q.push(neighbor);
        }
    }
}
```

الطوابير مزدوجة الأطراف (Deques)

🎯 المفهوم الأساسي

تعريف Deque

Deque = Double-Ended **Queue**

النطق الصحيح: "ديك" (Deck) - مثل "Deck of Cards"

المبدأ الأساسي

حرية كاملة من الطرفين!

- الإدخال: من الأمام **أو** من الخلف
- الحذف: من الأمام **أو** من الخلف

تشبيه بسيط 🚇

تخيل **نفق مفتوح من الجهتين**:

- القطارات تقدر تدخل من اليمين أو اليسار
- القطارات تقدر تطلع من اليمين أو اليسار
- **حرية مطلقة** في الحركة!

📊 جدول المقارنة الشامل

Stack vs Queue vs Deque

الميزة	Stack	Queue	Deque
المبدأ	LIFO	FIFO	Flexible
الإدخال	Top	Rear	Front & Rear
الحذف	Top	Front	Front & Rear
العمليات O(1)	✅	✅	✅
المرونة	محدود	محدود	عالي جداً ✨
التطبيقات	Undo, Back	Print, BFS	Sliding Window
Adapter كـ	–	–	يُنفذ Stack & Queue ✅

📋 العمليات الأساسية (Deque Operations)

العملية	الوصف	التعقيد الزمني	الاستثناء
`insertFront(e)`	إضافة عنصر في الأمام	O(1)	–
`insertBack(e)`	إضافة عنصر في الخلف	O(1)	–
`()eraseFront`	حذف العنصر الأمامي	O(1)	DequeEmpty
`()eraseBack`	حذف العنصر الخلفي	O(1)	DequeEmpty
`()front`	عرض العنصر الأمامي	O(1)	DequeEmpty
`()back`	عرض العنصر الخلفي	O(1)	DequeEmpty
`()size`	عدد العناصر	O(1)	–
`()empty`	هل فارغ؟	O(1)	–

مثال توضيحي

```cpp
Deque d;  // فارغ

insertBack(10)    → [10]
insertBack(20)    → [10, 20]
insertFront(5)    → [5, 10, 20]
insertBack(30)    → [5, 10, 20, 30]
insertFront(1)    → [1, 5, 10, 20, 30]

front()           → 1
back()            → 30

eraseFront()      → [5, 10, 20, 30]
eraseBack()       → [5, 10, 20]
```

🔗 التنفيذ الأمثل: Doubly Linked List

لماذا Doubly Linked List؟

مقارنة الخيارات

التنفيذ	insertFront	insertBack	eraseFront	eraseBack
Simple Array	❌ O(n)	O(1)	❌ O(n)	O(1)
Singly Linked	O(1)	O(1)	O(1)	❌ O(n)
Doubly Linked	✅ O(1)	✅ O(1)	✅ O(1)	✅ O(1)

🃏 Deque كـ Adapter Pattern

المفهوم الذهبي 💡

Deque = الجوكر = يقدر يتحول لأي شي!

Deque يقدر يشتغل كـ:

1. **Stack** (مكدس)
2. **Queue** (طابور)

1. تنفيذ Stack باستخدام Deque

عملية Stack	نستخدم من Deque
`push(e)`	`insertFront(e)`
`pop()`	`eraseFront()`
`top()`	`front()`

2. تنفيذ Queue باستخدام Deque

عملية Queue	نستخدم من Deque
`enqueue(e)`	`insertBack(e)`
`dequeue()`	`eraseFront()`
`front()`	`front()`

الفرق بين ADT و STL

الميزة	Deque ADT	STL Deque
الإضافة (أمام)	`insertFront(e)`	`push_front(e)`
الإضافة (خلف)	`insertBack(e)`	`push_back(e)`
الحذف (أمام)	`eraseFront()`	`pop_front()`
الحذف (خلف)	`eraseBack()`	`pop_back()`
الاستثناء	يرمي `DequeEmpty`	⚠️ Undefined Behavior
الوصول العشوائي	–	`d[i]` ✨ (ميزة إضافية)